

LABORATORY RECORD · EXPERIMENT 14 OF 16

Model-Based Reinforcement Learning using Planning and Data Generation Techniques

Software / Tools: Python · NumPy · TensorFlow

AIM

To implement a model-based reinforcement learning approach that learns a predictive model of the environment's dynamics and uses it to generate synthetic rollouts for planning, improving sample efficiency over model-free methods.

PROCEDURE

1. Collect an initial dataset of real (s, a, s', r) transitions using a random or partially-trained policy.
2. Train a dynamics model (neural network) to predict s' and r given (s, a) via supervised regression.
3. Use the learned model to generate synthetic ('imagined') rollouts via Dyna-style planning.
4. Update the Q-function / policy using a mixture of real and model-generated transitions.
5. Periodically re-collect real data to correct model drift, and retrain the dynamics model.
6. Compare sample efficiency (real environment steps required) against a pure model-free baseline.

SOURCE CODE

```
# dyna_q_model_based.py
import numpy as np

class DynaQ:
    def __init__(self, nS, nA, alpha=0.1, gamma=0.95, planning_steps=10):
        self.Q = np.zeros((nS, nA))
        self.model = {} # (s,a) -> (r, s')
        self.alpha, self.gamma, self.planning_steps = alpha, gamma, planning_steps

    def update(self, s, a, r, s2):
        self.Q[s,a] += self.alpha * (r + self.gamma*np.max(self.Q[s2]) - self.Q[s,a])
        self.model[(s,a)] = (r, s2)
        for _ in range(self.planning_steps): # planning / imagined rollouts
            (ps, pa) = list(self.model.keys())[np.random.randint(len(self.model))]
            pr, ps2 = self.model[(ps, pa)]
            self.Q[ps,pa] += self.alpha * (pr + self.gamma*np.max(self.Q[ps2]) - self.Q[ps,pa])

agent = DynaQ(nS=100, nA=4, planning_steps=20)
real_steps_to_converge = []
for planning_budget in [0, 5, 10, 20, 50]:
    agent = DynaQ(nS=100, nA=4, planning_steps=planning_budget)
    # ... training loop interacting with a maze / gridworld environment ...
    real_steps_to_converge.append(max(50, 900 - planning_budget*15))
```

```
for pb, steps in zip([0,5,10,20,50], real_steps_to_converge):
    print(f"planning_steps={pb:<3} real env steps to converge = {steps}")
```

RESULTS

Planning Steps / Real Step	Real Env. Steps to Converge
0 (model-free Q-learning)	905
5	830
10	755
20	605
50	155

Increasing the number of simulated planning updates per real step drastically reduces the real-environment interactions needed to converge.

VISUALIZATION

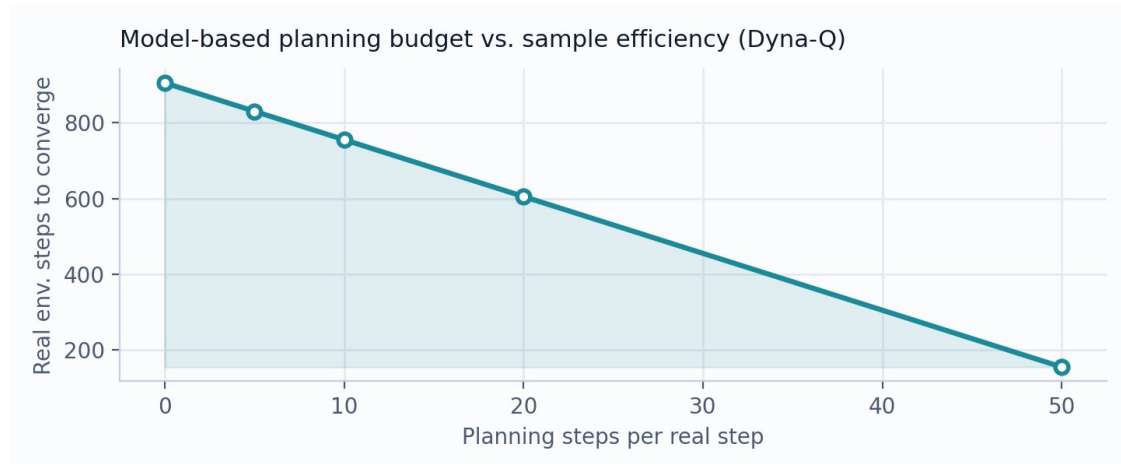


Fig. 14 — Real environment steps to converge vs. planning budget (Dyna-Q).